

BASA



(Shively, 2008)

EGB242 ASSIGNMENT 1

Signal Analysis

Rory Blair
n10773398

Introduction

As an intern at BASA, the first assignment is to analyse and complete the configuration for multiple elements of the communication system. This task will require the retrieval of audible dialogue from noisy audio created by equipment. It will then require the audible dialogue to be sent through a channel, simulating the transmission process, whilst still outputting audible dialogue. The final requirement of this task will be to convert the transmitted audio from its original analogue signal to a digital signal.

This Report will explain the tasks undertaken to reverse the additive noise created by multiple pieces of equipment by modelling the noise signal as a piecewise function. After audible dialogue is awarded, this report will then present the stages taken to simulate transmission and the conversion of an analogue signal to a digital signal.

1). Reversing Additive Noise

An inspection of the audio signal created by the recently installed microphone at mission control has concluded. It has been perceived that some hardware near the microphone has created a disruption in audio quality caused by a periodic signal distributed from nearby hardware. This portion of the report will describe the process to reverse additional noise created by nearby equipment.

The noise signal ([Figure 1](#)) will be shaped as a piecewise function and the additive noise model ([Appendix 3](#)) will be reversed to make audible dialogue. The dialogue that should have been sent through the microphone has come out as an incoherent sound, with minor snippets of coherent sound around every two seconds.

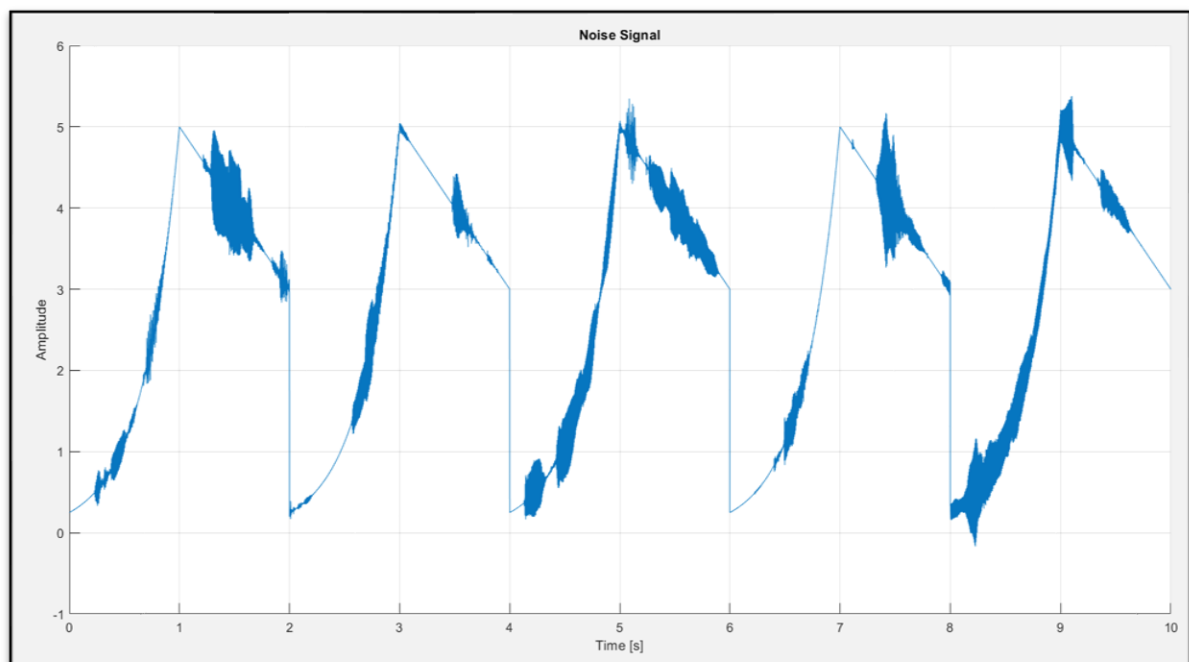


Figure 1: The Noise Signal

1.1). Complex Fourier Approximation

The periodic signal function distributed by mission control equipment has been modelled as a piecewise function which can be shown as follows:

$$n(t) = \begin{cases} 5e^{3(t-1)}, & 0 \leq t < 1 \\ -2t + 7, & 1 \leq t < 2 \end{cases}$$

The similarity between the periodic signal plotted and the piecewise function are seen when plotting the piecewise function. It shows a near match to one period of the periodic signal. (0 to 2, 2 to 4, 4 to 6, 6 to 8 and 8 to 10 of periodic signal) After the function of the periodic signal has been found, it is able to be used to create a complex Fourier series approximation ([Below](#)), which would be then used to reverse the additive noise model ([Appendix 3](#)).

$$nApproximation(t) = C_0 + \sum_{\substack{n=-\infty \\ n \neq 0}}^{\infty} C_n \times e^{2j\pi n f_0 t}$$

To start off, the values of ‘ C_0 ’ and ‘ C_n ’ need to be computed by hand ([Appendix 1](#)). Using these values, we would then use the complex Fourier Series equation to create an approximation equation of the noise signal, where ‘ n ’ is the number of harmonics used in this particular approximation. The summation would sum all values from negative ‘ n ’ to positive ‘ n ’ however, it would exclude values where ‘ n ’ is equal to zero as e^0 would equal one thus being useless. This is why ‘ C_0 ’ is outside of the summation. Finding the noise approximation signal via MATLAB can be seen below. A table of all coefficients of ‘ C_n ’ from negative five to positive five, excluding zero can be seen in [Appendix 8](#).

43	<code>%% 1.5 %%</code>	
44	<code>N = 5; % Harmonics</code>	
45	<code>ns = -N:N; % Harmonic Range</code>	
46	<code>Cn = zeros(1, 2*N+1); %Create 'Cn' vector</code>	
47	<code>for n = ns % Compute Numbers for different values of 'N'</code>	
48	<code> Cn(ns == n) = (1/T) * (((5*exp(-1i*pi*n))-(5*exp(-3)))/(3-(1i*pi*n))) + ((((3i*pi*n)-2)*(exp(-2i*pi*n)))-(((5i*pi*n)-2)*(exp(-1i*pi*n)))) / ((pi^2) * (n^2)));</code>	
49	<code>end</code>	
50	<code>c0 = (1/T) * (((5*exp(3))-5)/(3*exp(3)))+4; % Calculated value of 'c0'</code>	
51	<code>Cn(ns == 0) = c0; % Put in 'c0', as the 'for' loop has created an error during the 6th loop</code>	
52		
53	<code>%% 1.6 %%</code>	
54	<code>nApprox = zeros(size(t)); % Created 'nApprox' vector</code>	
55	<code>for n = ns % Summation of Cn Values</code>	
56	<code> nApprox = nApprox + Cn(ns == n) * exp(2i*pi*n*f0*t);</code>	
57	<code>end</code>	

1.2). Removing Noise from Original Audio

The approximation found in [Section 1.1](#). will now be used to reverse the additive noise process ([Appendix 3](#)). Firstly, the current approximation is between zero and two seconds, whilst the original audio is ten seconds long. This means that this approximation needs to be repeated five times to reverse the entire additive noise process ([Figure 2](#)). This is also done via MATLAB using the 'repmat' function to repeat the current approximation five times.

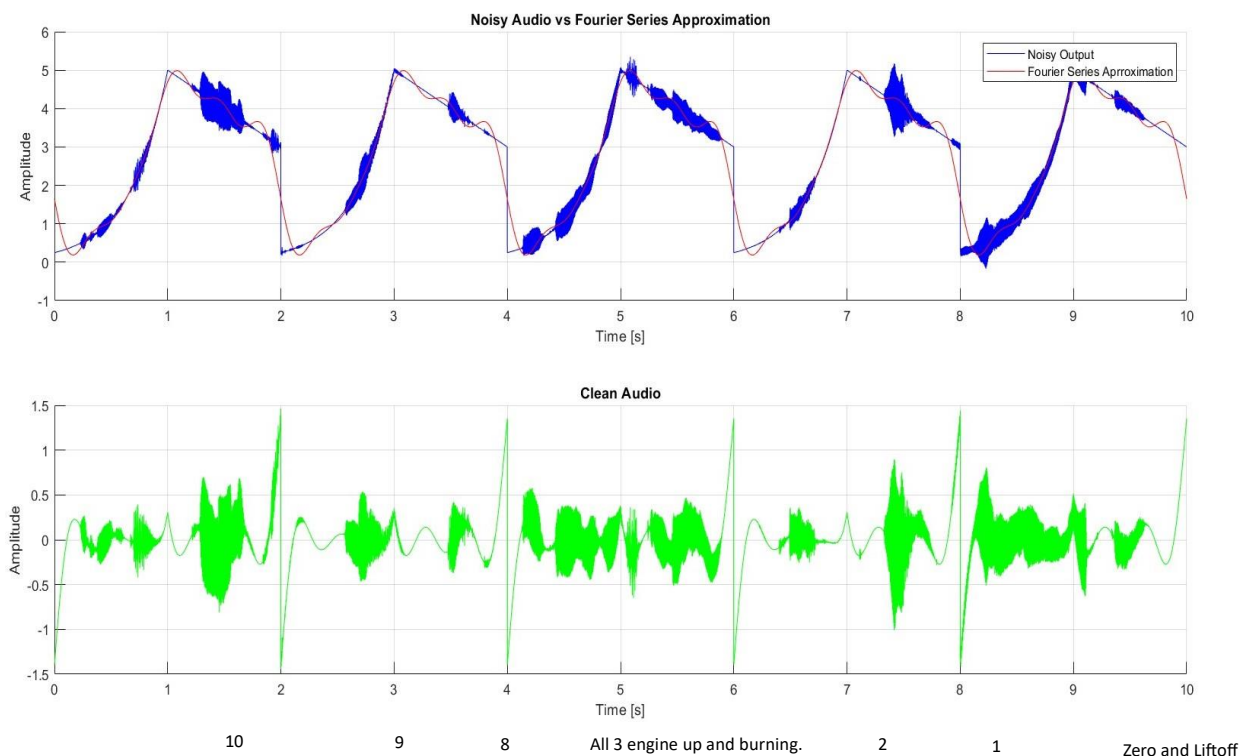


Figure 2: Noise Signal vs Approximation (Top), Clean Audio (Bottom)

After this process is complete, we are able to simply subtract the noise audio by the 'ten second approximation', which results in a somewhat clean audio with understandable dialogue ([Figure 2](#)). As seen above the approximation isn't exact but is sufficient to decrypt the noise from the audio. The only noise remaining is present when the voice is also present.

1.3). Harmonics

The number of harmonics used to make the approximation can produce results with varying levels of accuracy. When the number of harmonics ('N') is less than five, this results in the approximation being less accurate than the original. When the number of harmonics is more than five the result will be more accurate than the original. The clean audio plot also appears to be cleaner the higher the number of harmonics used. ([Appendix 4](#)). Although the accuracy of the approximation depends on the number of harmonics, the speed at which it can be

calculated is also dependant on the number of harmonics used. Any number of harmonics above 15 seems to not change the accuracy of the approximation. This would indicate that this number of harmonics results in a sufficiently accurate approximation which is able to be computed via MATLAB.

2). Transmitting/Receiving Signals

With the coherent audio, the signal is now ready to simulate the transmission from mission control to the orbiter. In order to transfer the signal, it must be modulated. This segment of the report will outline the process taken to simulate the transmission via modulation of the signal. The clean audio from [Section 1](#) graphed in the frequency domain shown below ([Figure 3](#)).

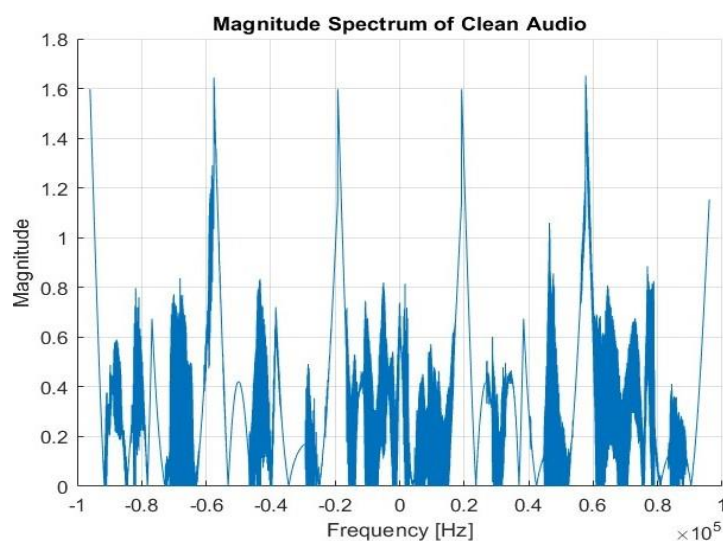


Figure 3: Clean Audio in Frequency Domain

2.1). Modulation and Demodulation

This process uses a function given to us called 'channel', which simulates the transmission from mission control to the orbiter. Channel takes a student ID and a variable named 'input' which is a row vector and with these two variables the channel function produces a quiet but audible audio which sounds like music. To modulate the audio, the audio itself must be plotted and analysed in the frequency domain to find an empty space where no spikes are present. This can be done in MATLAB by plotting the absolute values of 'channelQuietShift', a Fourier transform of 'channelQuiet', against the frequency domain. ([Figure 4](#))

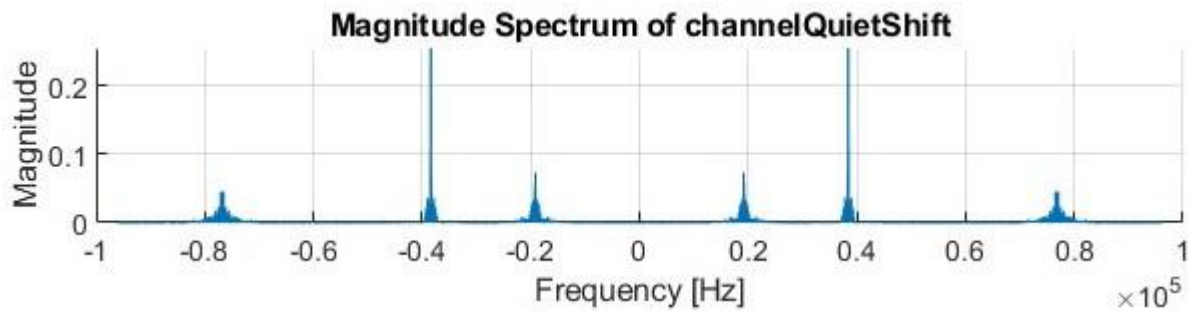


Figure 4: Absolute Values of 'channelQuietShift'

As seen in above (Figure 4), there is a gap at approximately sixty thousand Hertz, the carrier frequency that is required. Using this carrier frequency, the signal can now be modulated, using the carrier signal provided ($\cos(2\pi f_c t)$) and multiplying it with the clean audio, as seen in MATLAB code Below:

123	%% 2.4 %%
124	<code>fc = 60000; % Chosen as it is where there is space with no noise present</code>
125	<code>TF = samplesf / fs; % Create 'TF' Period</code>
126	<code>tf = linspace(0, TF, samplesf + 1); tf(end) = []; % Create 'tf' Vector</code>
127	<code>tone = audioClean .* cos(2*pi*fc*tf); % Modulate</code>
128	

After this process, the 'tone' variable, seen above, is put into the channel function, which results in a similar magnitude spectrum, but with a spike at around sixty thousand hertz.(Appendix 5), This would be the clean audio going through the transmission simulation. Now the audio signal is demodulated by modulating the 'channelled tone' (output) via multiplication between 'output' and the carrier signal ($\cos(2\pi f_c t)$). Next, the demodulated signal is put through a low pass filter, via MATLAB's 'lowpass' function, to remove other spikes from the graph thus removing extra noise. As seen in Figure 5, all noise has been removed except the sound required which is the original clean audio.

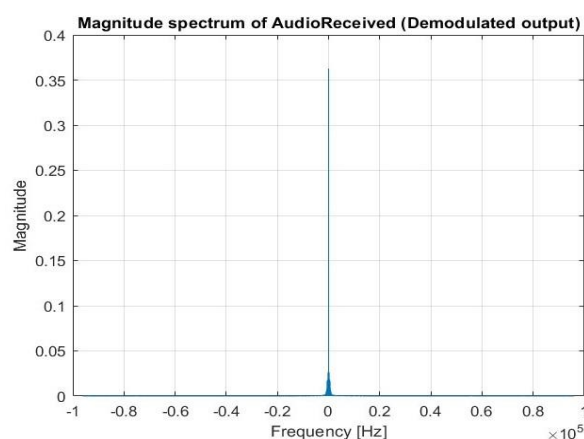


Figure 5: Absolute Values of Demodulated Audio

2.2). Investigation of demodulation with other Signals

After getting our original clean audio through the transmission simulator without problems, the other spikes seen in [Figure 4](#) can now be investigated. These other spikes are at nineteen thousand Hertz, thirty-eight thousand Hertz and seventy-six thousand Hertz. To find what these noises are it is required that modulation is performed using 'channelQuiet' using the carrier signal and substituting the three values in for the carrier frequency. Next the low pass filter function is used to remove other noises except at the carrier frequency. The first noise, at nineteen thousand Hertz, is a section of 'Lemonade' by 'SOPHIE' ([Xeon, 2014](#)). The Second noise, at thirty-eight thousand Hertz, is the part of the original 'Halo Theme' song from 2002 ([Salvatori, 2001](#)). The third and final noise, at seventy-six thousand Hertz, seems to be a very noise rendition of the beginning of 'Through the Fire and Flames' by 'Dragon Force' ([DragonForce, 2005](#)).

3). Analog-to-Digital Conversion

With the demodulation process completed, allowing transmissions to be heard via speakers, it doesn't allow the transmission to be heard via headphones. This is due to the fact that the speaker takes in analogue signals, whilst the headphones take in digital signals, which are crucial for communication between astronauts inside and outside the orbiter. To correct this problem, a conversion from analogue to digital signals must be performed in order for effective communication. This conversion will be done via the procedure of quantisation of the audio. The final section of this report describes the actions undertaken to convert the original analogue signal to a digitised version.

3.1). Resampling Audio

Initially, the demodulated audio is resampled to a new sample rate to adhere to the valid sampling rates supported by computer systems ([Appendix 6](#)). The new sampling rate was forty-four thousand one hundred Hertz as it was the closest to the Nyquist rate calculated. This resampled audio has been sampled to a lower rate which resulted in it being noisier. The resampled audio sounds clear and coherent but has some additional noise constantly in the background. The audio was also normalised to make quantisation easier in the next part of the process.

3.2). Quantise Audio

Next the resampled audio is quantised by using mid-treads or mid-risers. The normalised audio makes it easier to quantise as the maximum and minimum amplitude are now set to positive one and negative one, as opposed to finding them via a graph ([Appendix 7](#)). The number of quantisation levels is sixteen and both mid-treads and mid-risers are then tested. This process is done in MATLAB via the following code.

```

196 %% 3.3 %%
197 xmax = 1; xmin = -1; % Create xmin and xmax
198 L = 16; delta = (xmax - xmin) / L; % 'L' is Levels
199 input = linspace(xmin, xmax, 1000); % create input vector
200 audioQuantisedMT = delta * floor((AudioNormalised/delta) + (1/2)); % Create Mid-Tread
201 audioQuantisedMT(audioQuantisedMT >= xmax) = xmin + delta*(L-1);
202 audioQuantisedMR = delta * (floor(AudioNormalised/delta) + (1/2)); % Create Mid-Riser
203 audioQuantisedMR(audioQuantisedMR >= xmax) = xmin + delta*(L-(1/2));

```

Once it has been computed and the audio is now ready to be heard. The audio has deteriorated in quality, which is expected as the new sampling rate (fs2) makes sure that the resamples audio is at a valid sampling rate, in this case less Hertz. Although it is noisier, most dialogue is still comprehensible. However, as both mid-tread and mid-riser processes were tested, the mid-tread appears to have less noise, which would make it the best option for this mission. The mid-riser also seems to have an extra-long tread after zero (Figure 6).

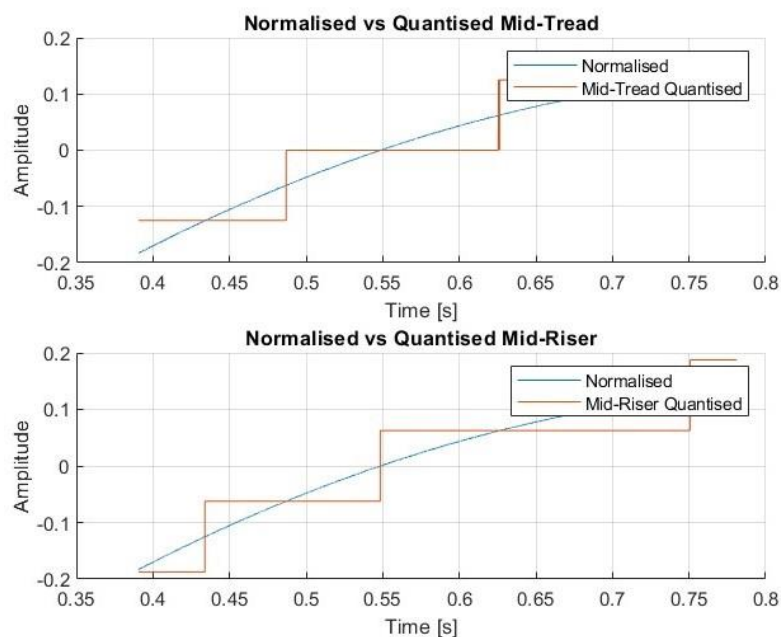


Figure 6: Mid-Tread vs Normalised(Top), Mid-Riser vs Normalised(Bottom)

3.3). Examination of Different Quantisation Levels

An inspection into the effect of different quantisation levels was undertaken. The results suggest that they act similar to the harmonics from Section 1. The less levels there are, the less accurate the mid-treads and mid-risers are to the normalised audio, thus the audio is less coherent. The more levels there are, the more accurate the mid-treads and mid-risers are to the normalised audio, thus a more coherent audio. This can be shown in Figure 7 below. However, if the quantisation level was five hundred the mid-tread and mid-riser would like very similar to the normalised audio (Appendix 9)

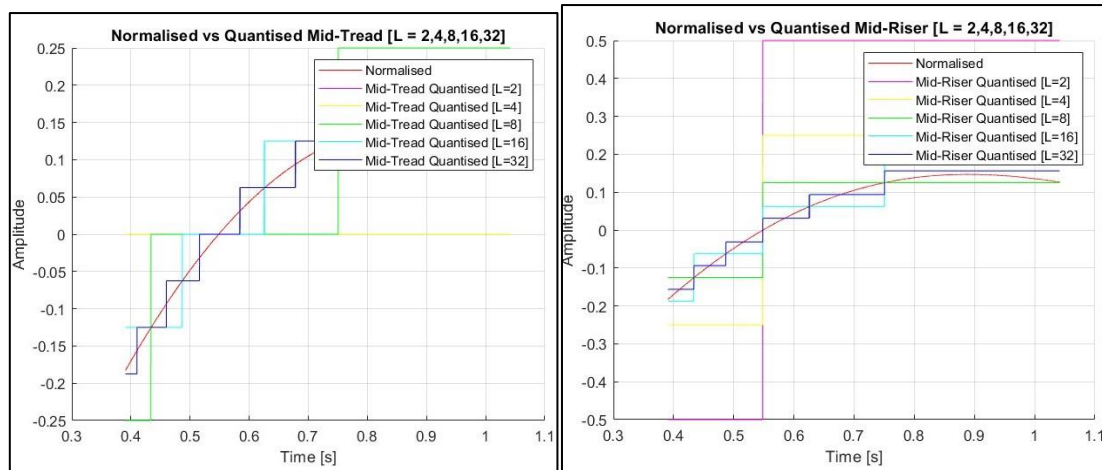


Figure 7: Mid-Tread Values(Left), Mid-Riser Values(Right)

Conclusion

In conclusion, multiple elements of the communication system, including the microphone, radio transmitter, radio receiver and ‘analog-to-digital’ converter, were analysed. The microphone was found to have a periodic signal, from nearby equipment, disrupting the audio quality. The radio transmitter sending audio and the radio receiver was found to be receiving garbled audio. The astronaut’s headphones were found to only take digital signals, whilst the signals being transmitted were analogue signals. These processes herein were configured to make corrections to these original problems. This vital system has gone from a barely working transmission array to a very effective communication system, which is ready to be implemented into the MARS-242 program.

Reflection

My understanding of the concepts used in this assignment have been demonstrated via this report. This report has been a three part process on how to get rid of a known periodic signal, to being able to decipher a simulated transmission, and then being able to convert analogue signals to digital signals. My grasp on concepts used is also shown by explaining what needs to be done, why it needs to be done and explaining plots and pieces of code.

A lesson learnt from doing this assignment is that I need to communicate more often, with tutors, lecturer and/or other students so that I have someone to bounce ideas off. This would help me in the future as it would be way to see if I am on the right track or not. Another lesson I learnt from this assignment was that I should ask more precise question when asking a tutor and not leave my questions to the last minute before a tutor has to leave. This happened one time and I felt stupid for not pushing to ask the question earlier. If I was to do this assignment again I would take the lessons learned here and use them as to get more clarification on certain concepts earlier.

References

DragonForce, 2005. *Through the Fire and Flames*. [Sound Recording] (Sam Totman, Herman Li, Vadim Pruzhanov).

QUT, 2023. *EGB242 - Assignment 1 (25%)*. [Online]
Available at: https://canvas.qut.edu.au/courses/2723/pages/assignment-1-problem-solving-task-details?module_item_id=836429
[Accessed 23 March 2023].

Salvatori, M. O. a. M., 2001. *Halo Theme*. [Sound Recording].

Shively, M. J., 2008. *Signals Analysis Squadron (AFISRA)*. [Online]
Available at: <https://www.afhra.af.mil/About-Us/Fact-Sheets/Display/Article/432387/signals-analysis-squadron-afisra/>
[Accessed 19 04 2023].

Xeon, S., 2014. *Lemonade*. [Sound Recording] (SOPHIE).

Appendix

Appendix 1: Section 1.4 Hand-Working

$$n(t) = \begin{cases} 5e^{3(t-1)}, & 0 \leq t < 1 \\ -2t + 7, & 1 \leq t < 2 \end{cases} \quad T = 2, \quad t_0 = 0, \quad f_0 = \frac{1}{T} = \frac{1}{2}$$

$$C_0 = \frac{1}{T} \left(\int_{t_0}^{\frac{T}{2}} 5e^{3(t-1)} dt + \int_{\frac{T}{2}}^T (-2t + 7) dt \right) = \frac{1}{2} \left(\int_0^1 5e^{3(t-1)} dt + \int_1^2 (-2t + 7) dt \right)$$

$$\int 5e^{3(t-1)} dt = \frac{5e^{3(t-1)}}{3}$$

$$\begin{aligned} \int_0^1 5e^{3(t-1)} dt &= \left[\frac{5e^{3(t-1)}}{3} \right]_0^1 \\ &= \left(\frac{5e^{3(1-1)}}{3} \right) - \left(\frac{5e^{3(0-1)}}{3} \right) \\ &= \frac{5e^{3 \times 0}}{3} - \frac{5e^{3 \times (-1)}}{3} = \frac{5e^0}{3} - \frac{5e^{-3}}{3} \\ &= \frac{5}{3} - \frac{5}{3e^3} = \frac{5e^3}{3e^3} - \frac{5}{3e^3} \\ &= \frac{5e^3 - 5}{3e^3} \approx 1.58368 \dots \end{aligned}$$

$$\int (-2t + 7) dt = \frac{-2t^2}{2} + 7t = -t^2 + 7t$$

$$\begin{aligned} \int_1^2 (-2t + 7) dt &= [-t^2 + 7t]_1^2 \\ &= (-2^2 + (7 \times 2)) - (-1^2 + (7 \times 1)) \\ &= (-4 + 14) - (-1 + 7) \\ &= 10 - 6 \\ &= 4 \end{aligned}$$

$$C_0 = \frac{1}{2} \left(\int_0^1 5e^{3(t-1)} dt + \int_1^2 (-2t + 7) dt \right) = \frac{1}{2} \left(\frac{5e^3 - 5}{3e^3} + 4 \right) \approx 2.79184$$

$$\begin{aligned}
C_n &= \frac{1}{T} \left(\int_{t_0}^{\frac{T}{2}} 5e^{3(t-1)} e^{-2j\pi n f_0 t} dt + \int_{\frac{T}{2}}^T (-2t+7) e^{-2j\pi n f_0 t} dt \right) \\
&= \frac{1}{2} \left(\int_0^1 5e^{3(t-1)} e^{-2j\pi n 0.5t} dt + \int_1^2 (-2t+7) e^{-2j\pi n 0.5t} dt \right) \\
&= \frac{1}{2} \left(\int_0^1 5e^{3(t-1)} e^{-j\pi n t} dt + \int_1^2 (-2t+7) e^{-j\pi n t} dt \right) \\
\int 5e^{3(t-1)} e^{-j\pi n t} dt &= \int 5e^{3(t-1)-j\pi n t} dt \\
u = 3(t-1) - j\pi n t &= 3t - 3 - j\pi n t \rightarrow du = 3 - j\pi n \\
\int 5e^{3(t-1)-j\pi n t} dt &= \frac{-5}{j\pi n - 3} \int e^u du \\
&= \frac{-5e^u}{j\pi n - 3} \\
&= \frac{-5e^{3(t-1)-j\pi n t}}{j\pi n - 3} \\
&= \frac{5e^{3(t-1)-j\pi n t}}{3 - j\pi n}
\end{aligned}$$

$$\begin{aligned}
\int_0^1 5e^{3(t-1)-j\pi n t} dt &= \left[\frac{5e^{3(t-1)-j\pi n t}}{3 - j\pi n} \right]_0^1 \\
&= \left(\frac{5e^{3(1-1)-j\pi n 1}}{3 - j\pi n} \right) - \left(\frac{5e^{3(0-1)-j\pi n 0}}{3 - j\pi n} \right) \\
&= \left(\frac{5e^{(3 \times 0) - (j\pi n \times 1)}}{3 - j\pi n} \right) - \left(\frac{5e^{(3 \times (-1)) - (j\pi n \times 0)}}{3 - j\pi n} \right) \\
&= \left(\frac{5e^{-j\pi n}}{3 - j\pi n} \right) - \left(\frac{5e^{(-3)}}{3 - j\pi n} \right) \\
&= \frac{5e^{-j\pi n} - 5e^{-3}}{3 - j\pi n}
\end{aligned}$$

$$\int (-2t + 7) e^{-j\pi nt} dt = \int (7 - 2t) e^{-j\pi nt} dt = - \int (2t - 7) e^{-j\pi nt} dt$$

$$\int u \times v' = u \times v - \int u' \times v,$$

$$u = 2t - 7, v' = e^{-j\pi nt}$$

$$u' = 2, v = \frac{-e^{-j\pi nt}}{j\pi n} = \frac{je^{-j\pi nt}}{\pi n}$$

$$\int (2t - 7) e^{-j\pi nt} dt = \left((2t - 7) \times \frac{je^{-j\pi nt}}{\pi n} \right) - \int \left(2 \times \frac{je^{-j\pi nt}}{\pi n} \right)$$

$$\int \left(2 \times \frac{je^{-j\pi nt}}{\pi n} \right) dt = -\frac{2}{\pi^2 n^2} \int e^u du, \quad \text{where } u = -j\pi nt, du = -j\pi n dt$$

$$= -\frac{2e^u}{\pi^2 n^2}$$

$$= -\frac{2e^{-j\pi nt}}{\pi^2 n^2}$$

$$\int (2t - 7) e^{-j\pi nt} dt = \frac{j(2t - 7)e^{-j\pi nt}}{\pi n} - \left(-\frac{2e^{-j\pi nt}}{\pi^2 n^2} \right)$$

$$= \frac{j(2t - 7)e^{-j\pi nt}}{\pi n} + \frac{2e^{-j\pi nt}}{\pi^2 n^2}$$

$$- \int (2t - 7) e^{-j\pi nt} dt = -\frac{j(2t - 7)e^{-j\pi nt}}{\pi n} - \frac{2e^{-j\pi nt}}{\pi^2 n^2}$$

$$\int (7 - 2t) e^{-j\pi nt} dt = -\frac{j\pi n(2t - 7)e^{-j\pi nt}}{\pi^2 n^2} - \frac{2e^{-j\pi nt}}{\pi^2 n^2}$$

$$= -\frac{j\pi n(2t - 7)e^{-j\pi nt} - 2e^{-j\pi nt}}{\pi^2 n^2}$$

$$= \frac{(-j\pi n(2t - 7) - 2)e^{-j\pi nt}}{\pi^2 n^2}$$

$$= \frac{-(j\pi n(2t - 7) + 2)e^{-j\pi nt}}{\pi^2 n^2}$$

$$= \frac{-(2j\pi nt - 7j\pi n + 2)e^{-j\pi nt}}{\pi^2 n^2}$$

$$\begin{aligned}
\int_1^2 (7-2t)e^{-j\pi nt} dt &= \left[\frac{-(2j\pi nt - 7j\pi n + 2)e^{-j\pi nt}}{\pi^2 n^2} \right]_1^2 \\
&= \left(\frac{-((2j\pi n \times 2) - 7j\pi n + 2)e^{-j\pi n \times 2}}{\pi^2 n^2} \right) - \left(\frac{-((2j\pi n \times 1) - 7j\pi n + 2)e^{-j\pi n \times 1}}{\pi^2 n^2} \right) \\
&= \left(\frac{-(4j\pi n - 7j\pi n + 2)e^{-2j\pi n}}{\pi^2 n^2} \right) - \left(\frac{-(2j\pi n - 7j\pi n + 2)e^{-j\pi n}}{\pi^2 n^2} \right) \\
&= \left(\frac{(-4j\pi n + 7j\pi n - 2)e^{-2j\pi n}}{\pi^2 n^2} \right) - \left(\frac{(-2j\pi n + 7j\pi n - 2)e^{-j\pi n}}{\pi^2 n^2} \right) \\
&= \left(\frac{(3j\pi n - 2)e^{-2j\pi n}}{\pi^2 n^2} \right) - \left(\frac{(5j\pi n - 2)e^{-j\pi n}}{\pi^2 n^2} \right) \\
&= \frac{(3j\pi n - 2)e^{-2j\pi n} - (5j\pi n - 2)e^{-j\pi n}}{\pi^2 n^2}
\end{aligned}$$

$$\begin{aligned}
C_n &= \frac{1}{2} \left(\int_0^1 5e^{3(t-1)} e^{-j\pi nt} dt + \int_1^2 (-2t + 7) e^{-j\pi nt} dt \right) \\
&= \frac{1}{2} \left(\frac{5e^{-j\pi n} - 5e^{-3}}{3 - j\pi n} + \frac{(3j\pi n - 2)e^{-2j\pi n} - (5j\pi n - 2)e^{-j\pi n}}{\pi^2 n^2} \right)
\end{aligned}$$

$$nApproximation(t) = C_0 + \sum_{\substack{n=-\infty \\ n \neq 0}}^{\infty} C_n \times e^{2j\pi n f_0 t}$$

$$Approx = \frac{1}{2} \left(\frac{5e^3 - 5}{3e^3} + 4 \right) + \sum_{\substack{n=-\infty \\ n \neq 0}}^{\infty} \frac{1}{2} \left(\frac{5e^{-j\pi n} - 5e^{-3}}{3 - j\pi n} + \frac{(3j\pi n - 2)e^{-2j\pi n} - (5j\pi n - 2)e^{-j\pi n}}{\pi^2 n^2} \right) \times e^{2j\pi n 0.5t}$$

$$Approx = \frac{1}{2} \left(\frac{5e^3 - 5}{3e^3} + 4 \right) + \sum_{\substack{n=-\infty \\ n \neq 0}}^{\infty} \frac{1}{2} \left(\frac{5e^{-j\pi n} - 5e^{-3}}{3 - j\pi n} + \frac{(3j\pi n - 2)e^{-2j\pi n} - (5j\pi n - 2)e^{-j\pi n}}{\pi^2 n^2} \right) \times e^{j\pi nt}$$

Appendix 2: Mission1A.m Code

```
%% EGB242 Assignment 1 %%
% This file is a template for your MATLAB solution.
%
% Before starting to write code, record your test audio with the record
% function as described in the assignment task.

%% Load recorded test audio into workspace
clear all; close all;
load DataA1;

% Begin writing your MATLAB solution below this line.

%% Section 1 %%
%% 1.1 %%
% n(t) =
% 5e^(3(t-1)), 0<=t<1
% -2t+7, 1<=t<2

%% 1.2 %%
% sound(audio, fs);

%% 1.3 %%
t0 = 0; % starting time
T = 2; % Period
f0 = 1/T; % Frequency
samples = fs * T; % Samples
t = linspace(t0, T, samples + 1); t(end) = []; % create 't' vector
repeat = 5;
tt = linspace(0, repeat * T, repeat * samples + 1); tt(end) = []; % create
'tt' vector
figure; hold on; grid on; %plot audio signal
plot(tt, audio);
title('Noise Signal');
xlabel('Time [s]');
ylabel('Amplitude');

%% 1.4 %%
% Done

%% 1.5 %%
N = 5; % Harmonics
ns = -N:N; % Harmonic Range
Cn = zeros(1, 2*N+1); %Create 'Cn' vector
for n = ns % Compute Numbers for different values of 'N'
    Cn(ns == n) = (1/T) * (( (5*exp(-1i*pi*n)) - (5*exp(-3))) / (3 - (1i*pi*n))
) + ( (((3i*pi*n)-2)*(exp(-2i*pi*n))) - (((5i*pi*n)-2)*(exp(-1i*pi*n)))) /
((pi^2) * (n^2)) ); % Calculated value of 'Cn'
end
c0 = (1/T) * (((5*exp(3))-5)/(3*exp(3))+4); % Calculated value of 'c0'
Cn(ns == 0) = c0; % Put in 'c0', as the 'for' loop has created an error
during the 6th loop
```

```

%% 1.6 %%
nApprox = zeros(size(t)); % Created 'nApprox' vector
for n = ns % Summation of Cn Values
    nApprox = nApprox + Cn(ns == n) * exp(2i*pi*n*f0*t);
end
figure; hold on; grid on;
plot(t, nApprox, 'b');
title('Fourier Series Approximation');
xlabel('Time [s]');
ylabel('Amplitude');
s1 = repmat(nApprox, 1, repeat);

%% 1.7 %%
audioClean = audio - real(s1);
figure;
subplot(2,1,1); hold on; grid on;
plot(tt, audio, 'b'); hold on;
plot(tt, s1, 'r');
title('Noisy Audio vs Fourier Series Approximation');
xlabel('Time [s]');
ylabel('Amplitude');
legend('Noisy Output', 'Fourier Series Aprroximation')
subplot(2,1,2); hold on; grid on;
plot(tt, audioClean, 'g');
title('Clean Audio');
xlabel('Time [s]');
ylabel('Amplitude');
% sound(audioClean, fs);

%% 1.8 %%
% Less Harmonics means the approximation is less accurate to the Noise
% Signal, n(t). At One Harmonic there is still noise but the dialogue is
% auditable.
% More Harmonics means the approximation more accurate, nearly perfect to
% the Noise Signal, n(t). When it is 20 or more harmonics is when it starts
% to become more accurate to n(t). (Appendix)

%% Section 2 %%
%% 2.1 %%
samplesf = length(audioClean); % create frequency sample
f = linspace(-fs/2, fs/2, samplesf + 1); f(end) = []; % create 'f' vector
figure; hold on; grid on; % plot against the f
plot(f,abs(audioClean))
title('Magnitude Spectrum of Clean Audio');
xlabel('Frequency [Hz]');
ylabel('Magnitude');

%% 2.2 %%
channelQuiet = channel(10773398, zeros(size(tt))); % Create channelQuiet
using channel function
channelQuietShift = fftshift(fft(channelQuiet)) / fs; % Create frequency
version of channelQuiet called channelQuietShift

```

```

%% 2.3 %%
figure;
subplot(3,1,1) % Plot channel Quiet in time
plot(tt,channelQuiet); hold on; grid on;
title('channelQuiet');
xlabel('Time [s]');
ylabel('Amplitude');
subplot(3,1,2); hold on; grid on;
plot(f, abs(channelQuietShift)) % Plot Magnitude Spectrum of
channelQuietShift in frequency
title('Magnitude Spectrum of channelQuietShift');
xlabel('Frequency [Hz]');
ylabel('Magnitude');
subplot(3,1,3); hold on; grid on;
plot(f, angle(channelQuietShift)) % Plot Phase Spectrum of
channelQuietShift in frequency
title('Phase Spectrum of channelQuietShift');
xlabel('Frequency [Hz]');
ylabel('Phase [rad]');

%% 2.4 %%
fc = 60000; % Chosen as it is where there is space with no noise present
TF = samplesf / fs; % Create 'TF' Period
tf = linspace(0, TF, samplesf + 1); tf(end) = []; % Create 'tf' Vector
tone = audioClean .* cos(2*pi*fc*tf); % Modulate

%% 2.5 %%
output = channel(10773398, tone); % Put new 'tone' through channel function
foutput = fftshift(fft(output)) / fs; % Create frequency version of
'output' called 'foutput'

figure;
subplot(2, 1, 1); hold on; grid on;
plot(f, abs(foutput)); % Plot Magnitude Spectrum of 'foutput'
title('Magnitude spectrum of foutput');
xlabel('Frequency [Hz]');
ylabel('Magnitude');
subplot(2, 1, 2); hold on; grid on;
plot(f, angle(foutput)); % Plot Phase spectrum of 'foutput'
title('Phase spectrum of foutput');
xlabel('Frequency [Hz]');
ylabel('Phase [rad]');

%% 2.6 %%
Cutf = 0.3 * (10^3); % This cut-off frequency was chosen to keep the
dialogue and rid of the noise
Dmod = output .* cos(2*pi*fc*tf); % Demodulate
audioReceived = lowpass(Dmod,Cutf,fs); % Put through a lowpass filter
AudioReceived = fftshift(fft(audioReceived)) / fs ; % Create frequency
version of 'audioReceived' called 'AudioReceived'
figure;
plot(f, abs(AudioReceived)); hold on; grid on; % Plot Magnitude Spectrum of
'AudioReceived'
title('Magnitude spectrum of AudioReceived (Demodulated output)');
xlabel('Frequency [Hz]');
ylabel('Magnitude');
% sound(audioReceived,fs)

```

```

%% 2.7 %%
Other = 19168.7; % The first spike near  $0.2 \times 10^5$ 
demodulateOther1 = channelQuiet .* cos(2*pi*tt.*Other); % modulate
filtered1 = lowpass(demodulateOther1,Cutf,fs); % filter through lowpass
% sound(filtered1, fs)
% Lemonade song - https://www.youtube.com/watch?v=LdLvp630plc
Other2 = 38370; % The second spike near  $0.4 \times 10^5$ 
demodulateOther2 = channelQuiet .* cos(2*pi*tt.*Other2); % modulate
filtered2 = lowpass(demodulateOther2,Cutf,fs); % filter through lowpass
% sound(filtered2, fs)
% Halo Theme
Other3 = 76702; % The third spike near  $0.8 \times 10^5$ 
demodulateOther3 = channelQuiet .* cos(2*pi*tt.*Other3); % modulate
filtered3 = lowpass(demodulateOther3,Cutf,fs); % filter through lowpass
% sound(filtered3, fs)
% Through The Fire And Flames By DragonForce

%% Section 3 %%
%% 3.1 %%
fs2 = 44100; %  $(1.7 \times (10^4)) \times 2$  and round up
AudioNormalised = audioReceived / max(abs(audioReceived)); % Normalise
Audio
audioResampled = resample(audioReceived, fs2, fs); % Resample audioReceived
AudioResampled = fftshift(fft(audioResampled)) / fs2; % Create frequency
version of audioResampled called 'AudioResampled'
qsamples = length(AudioNormalised); % quantise samples
q = linspace(0, qsamples / fs, samples + 1); t(end) = []; % Create qunatise
vector

figure; hold on; grid on; % Plot Received against Normalised
plot(tt,AudioNormalised);
plot(tt,audioReceived)
title('AudioNormalised vs audioReceived');
xlabel('Time [s]');
ylabel('Amplitude');
legend('Normalised', 'Received')

%% 3.2 %%
% sound(audioResampled, fs2)

```

```

%% 3.3 %%
xmax = 1; xmin = -1; % Create xmin and xmax
L = 16; delta = (xmax - xmin) / L; % 'L' is Levels
input = linspace(xmin, xmax, 1000); % create input vector
audioQuantisedMT = delta * floor((AudioNormalised/delta) + (1/2)); % Create
Mid-Tread
audioQuantisedMT(audioQuantisedMT >= xmax) = xmin + delta*(L-1);
audioQuantisedMR = delta * (floor(AudioNormalised/delta) + (1/2)); % Create
Mid-Riser
audioQuantisedMR(audioQuantisedMR >= xmax) = xmin + delta*(L-(1/2));
figure;
subplot(2,1,1); hold on; grid on;
plot(q(10000:20000), AudioNormalised(10000:20000));
plot(q(10000:20000), audioQuantisedMT(10000:20000));
title('Normalised vs Quantised Mid-Tread');
xlabel('Time [s]');
ylabel('Amplitude');
legend('Normalised', 'Mid-Tread Quantised')
subplot(2,1,2); hold on; grid on;
plot(q(10000:20000), AudioNormalised(10000:20000));
plot(q(10000:20000), audioQuantisedMR(10000:20000));
title('Normalised vs Quantised Mid-Riser');
xlabel('Time [s]');
ylabel('Amplitude');
legend('Normalised', 'Mid-Riser Quantised')
% sound(audioQuantisedMT, fs)
% sound(audioQuantisedMR, fs)

%% 3.4 %%
% L = 2
L2 = 2; delta2 = (xmax - xmin) / L2;
audioMT2 = delta2 * floor((AudioNormalised/delta2) + (1/2));
audioMT2(audioMT2 >= xmax) = xmin + delta2*(L2-1);
audioMR2 = delta2 * (floor(AudioNormalised/delta2) + (1/2));
audioMR2(audioMR2 >= xmax) = xmin + delta2*(L2-(1/2));
% L = 4
L4 = 4; delta4 = (xmax - xmin) / L4;
audioMT4 = delta4 * floor((AudioNormalised/delta4) + (1/2));
audioMT4(audioMT4 >= xmax) = xmin + delta4*(L4-1);
audioMR4 = delta4 * (floor(AudioNormalised/delta4) + (1/2));
audioMR4(audioMR4 >= xmax) = xmin + delta4*(L4-(1/2));
% L = 8
L8 = 8; delta8 = (xmax - xmin) / L8;
audioMT8 = delta8 * floor((AudioNormalised/delta8) + (1/2));
audioMT8(audioMT8 >= xmax) = xmin + delta8*(L8-1);
audioMR8 = delta8 * (floor(AudioNormalised/delta8) + (1/2));
audioMR8(audioMR8 >= xmax) = xmin + delta8*(L8-(1/2));
% L = 32
L32 = 32; delta32 = (xmax - xmin) / L32;
audioMT32 = delta32 * floor((AudioNormalised/delta32) + (1/2));
audioMT32(audioMT32 >= xmax) = xmin + delta32*(L32-1);
audioMR32 = delta32 * (floor(AudioNormalised/delta32) + (1/2));
audioMR32(audioMR32 >= xmax) = xmin + delta32*(L32-(1/2));

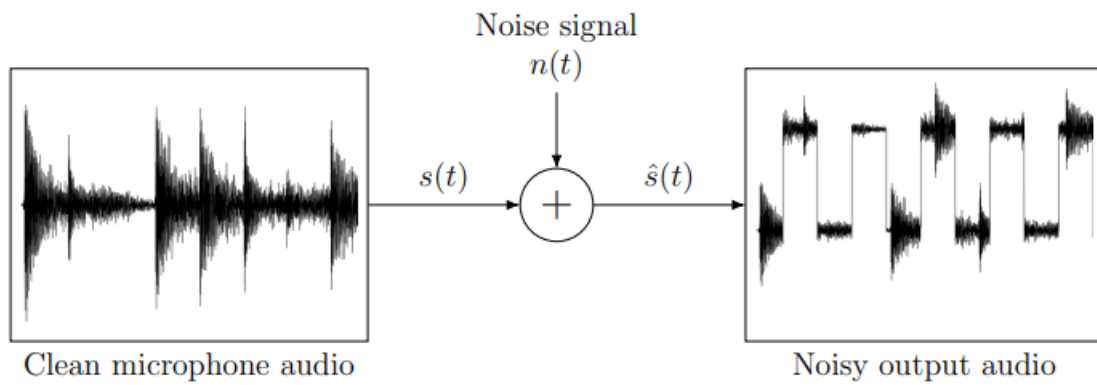
```

```

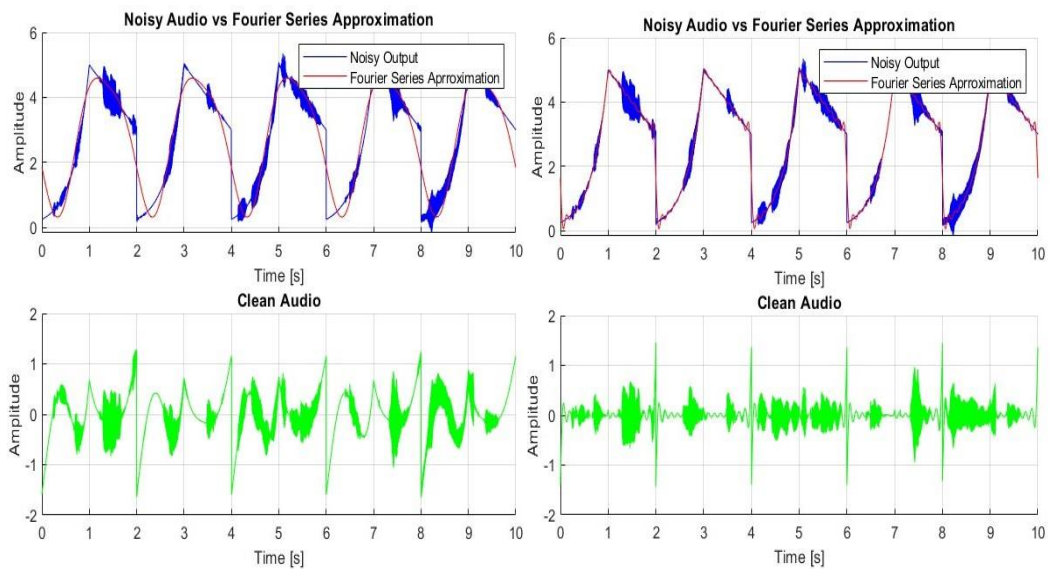
figure;hold on;grid on; % Plot Mid-Riser Tests
plot(q(15000:40000), AudioNormalised(15000:40000), 'r');
plot(q(15000:40000), audioMR2(15000:40000), 'm');
plot(q(15000:40000), audioMR4(15000:40000), 'y');
plot(q(15000:40000), audioMR8(15000:40000), 'g');
plot(q(15000:40000), audioQuantisedMR(15000:40000), 'c');
plot(q(15000:40000), audioMR32(15000:40000), 'b');
title('Normalised vs Quantised Mid-Riser [L = 2,4,8,16,32]');
xlabel('Time [s]');
ylabel('Amplitude');
legend('Normalised', 'Mid-Riser Quantised [L=2]', 'Mid-Riser Quantised [L=4]', 'Mid-Riser Quantised [L=8]', 'Mid-Riser Quantised [L=16]', 'Mid-Riser Quantised [L=32]')
figure;hold on;grid on; % Plot Mid-Tread Tests
plot(q(15000:40000), AudioNormalised(15000:40000), 'r');
plot(q(15000:40000), audioMT2(15000:40000), 'm');
plot(q(15000:40000), audioMT4(15000:40000), 'y');
plot(q(15000:40000), audioMT8(15000:40000), 'g');
plot(q(15000:40000), audioQuantisedMT(15000:40000), 'c');
plot(q(15000:40000), audioMT32(15000:40000), 'b');
title('Normalised vs Quantised Mid-Tread [L = 2,4,8,16,32]');
xlabel('Time [s]');
ylabel('Amplitude');
legend('Normalised', 'Mid-Tread Quantised [L=2]', 'Mid-Tread Quantised [L=4]', 'Mid-Tread Quantised [L=8]', 'Mid-Tread Quantised [L=16]', 'Mid-Tread Quantised [L=32]')

```

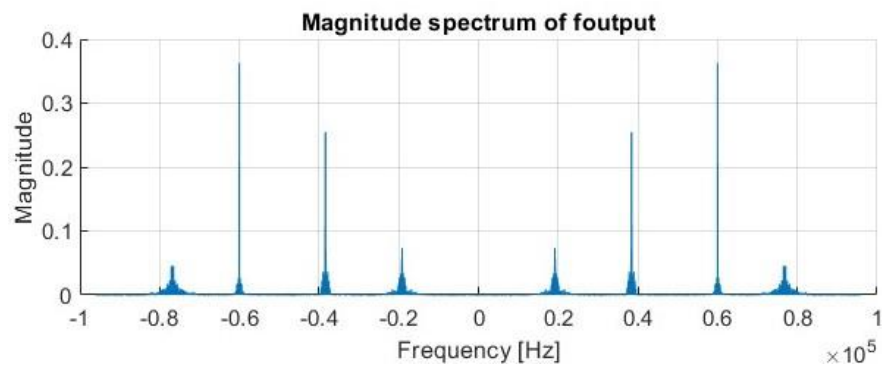
Appendix 3: Additive Noise Model (QUT, 2023)



Appendix 4: $N = 2$ (Left), $N = 15$ (Right)



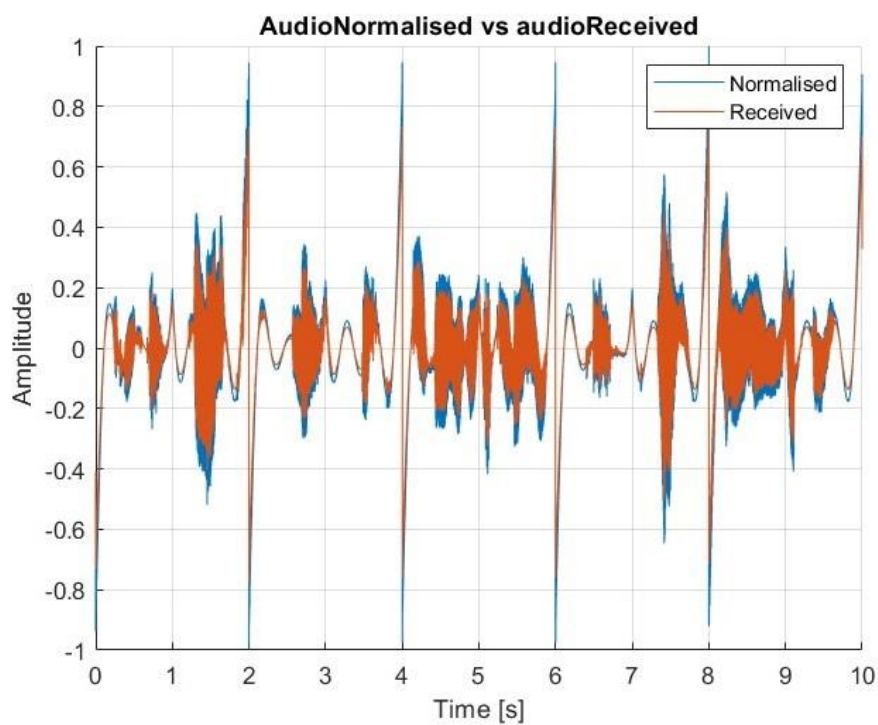
Appendix 5: Modulated Signal after channel



Appendix 6: Valid sampling rates, known to be widely supported by computer systems. (QUT, 2023)

Valid sampling rates [Hz]						
8000	11025	22050	44100	48000	96000	192000

Appendix 7: Normalised against received



Appendix 8: C_n Coefficients Table

n	Values
-5	-0.038892429755826 - 0.093448941102937i
-4	0.042696305038517 - 0.099268392779963i
-3	-0.102999203694150 - 0.171567170190313i
-2	0.147005561225258 - 0.148732784362849i
-1	-0.619895577548291 - 0.836293004718186i
1	-0.619895577548291 + 0.836293004718186i
2	0.147005561225258 + 0.148732784362849i
3	-0.102999203694150 + 0.171567170190313i
4	0.042696305038517 + 0.099268392779963i
5	-0.038892429755826 + 0.093448941102937i

Appendix 9: Quantisation level is one hundred.

